

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_09101709-554\agent-a5c11fc2fefe884e8.jsonl`
- SHA-256: `1c51f9e1777d8ebc16cd944e10536d4a2c160ed399674b2c41b234ec2e704217`
- Source modified: `2026-06-25T17:12:32+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_09101709-554`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T17:11:13.829Z)

PR #381 ".github/workflows/nightly.yml" is CHECKED OUT (branch chore/nightly-quality-regression). It adds a Tier-1 nightly quality-regression GitHub Actions workflow (schedule cron "0 3 * * *" + workflow_dispatch) that re-runs the repo's OFFLINE guards (ruff, mypy, run_all_tests --cov-fail-under=70, golden-fixtures regression). The ONLY changed file is .github/workflows/nightly.yml (105 lines, ci.yml untouched). CRITICAL CONTEXT: a schedule/workflow_dispatch workflow does NOT self-trigger on the PR, so the PR's green CI only YAML-PARSED nightly.yml ? its STEPS were never executed. So you must verify the steps would actually run. Read BOTH .github/workflows/nightly.yml AND .github/workflows/ci.yml to compare. Ground every claim in the files.

Review ONLY fidelity + runnability. (a) Does nightly.yml mirror ci.yml's OFFLINE lanes correctly ? same Python version pin, same per-lane pip installs (so modules don't skip for a missing dep), same exact commands incl. cov-fail-under=70? List any drift. (b) Since the PR CI never RAN these steps, verify each command is valid/runnable: actions/* versions (should be Node-24, e.g. checkout@v5+ not @v4), the python setup, and the commands. Pay special attention to any command NOT in ci.yml (e.g. python -m backend.eval.golden_fixtures --check) ? confirm the module + flag exist and the invocation is correct. (c) cron "0 3 * * *" valid? Report concrete findings with the exact line + a fix.

2. assistant (2026-06-25T17:11:16.193Z)

I'll review PR #381 for fidelity and runnability. Let me start by reading both workflow files.

3. user (2026-06-25T17:11:17.136Z)

```
1     name: Nightly quality regression (Tier-1, offline)
2
3     # Tier-1 nightly quality guard. Per-PR `ci.yml` only fires on push/pull_request;
4     # this runs the SAME offline guards on a schedule so we also catch what per-PR CI
5     # can't: dependency / transitive-dep drift, flakiness over time, and rot during
6     # quiet periods when nothing is pushed.
7     #
8     # Tier-1 == OFFLINE ONLY: no GPU, no real video, no `run_regression.py` on the
9     # real corpus (that's Tier-2 ? owner-gated, needs a self-hosted GPU runner +
10    # the gitignored golden corpus). The lanes here mirror `ci.yml`'s offline jobs
11    # verbatim (same Python pin, same per-lane pip installs) so modules collect/run
12    # instead of silently skipping for a missing dep.
13    #
14    # NOTE: scheduled workflows only fire from the repo's DEFAULT branch, so this
15    # won't actually run on a cadence until it lands on `master`. Use the
16    # `workflow_dispatch` trigger to run it on demand for validation after merge.
17
18    on:
19        schedule:
```

```

20     # 03:00 UTC daily.
21     - cron: "0 3 * * *"
22     workflow_dispatch:
23
24     permissions:
25         contents: read
26
27     concurrency:
28         group: nightly-`${{ github.ref }}`
29         cancel-in-progress: true
30
31     jobs:
32         # Lint gate (ruff.toml at repo root) ? mirrors ci.yml::lint.
33         lint:
34             name: Ruff lint
35             runs-on: ubuntu-latest
36             steps:
37                 - uses: actions/checkout@v6
38                 - uses: actions/setup-python@v6
39                   with:
40                       python-version: "3.13"
41                 - name: Install ruff
42                   run: python -m pip install ruff
43                 - name: Ruff check
44                   run: ruff check . --output-format github
45
46         # Type gate (mypy.ini at repo root) ? mirrors ci.yml::typecheck. Typed core
47         # deps are installed so mypy sees real signatures (without them
48         # ignore_missing_imports would Any-stub half the checks).
49         typecheck:
50             name: Mypy (lenient baseline)
51             runs-on: ubuntu-latest
52             steps:
53                 - uses: actions/checkout@v6
54                 - uses: actions/setup-python@v6
55                   with:
56                       python-version: "3.13"
57                 - name: Install mypy + typed core deps
58                   run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
59 genai numpy
60                 - name: Mypy
61                   run: mypy backend training
62
63         # Full offline suite + coverage floor ? mirrors ci.yml::full-suite. CPU-only
64         # torch (no CUDA wheels on a CPU runner), then the editable install pulls the
65         # rest. Floor stays at the ci.yml value (70); never lower it without an owner
66         # decision.
67         full-suite:
68             name: Full test suite (offline, mocked)
69             runs-on: ubuntu-latest
70             steps:
71                 - uses: actions/checkout@v6
72                 - uses: actions/setup-python@v6
73                   with:
74                       python-version: "3.13"
75                       cache: pip
76                 - name: System libs for opencv-python
77                   run: |
78                       sudo apt-get update
79                       sudo apt-get install -y libgl1 libglu1-mesa-dev libglu1-mesa
80 libgl1 libglu1-mesa-dev
81                 - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
82                   run: python -m pip install torch torchvision --index-url
83 https://download.pytorch.org/whl/cpu
84                 - name: Install package (editable) + dev tooling

```

```

82         run: python -m pip install -e .[dev]
83     - name: Run full suite (with coverage floor)
84         run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
85
86     # Offline quality guard: the committed golden regression fixtures
87     # (docs/GOLDEN_REGRESSION_FIXTURES.md) ? synthetic + de-attributed real
88     # trajectory fixtures, NO video / GPU / DB. Mirrors the offline replay in
89     # ci.yml::golden-crossos (single-OS here; ci.yml already covers cross-OS on
90     # every PR ? nightly is about drift-over-time on the default branch). The
91     # CONTROL replay needs only numpy.
92     golden-fixtures:
93         name: Golden fixtures regression (offline quality guard)
94         runs-on: ubuntu-latest
95         steps:
96             - uses: actions/checkout@v6
97             - uses: actions/setup-python@v6
98               with:
99                 python-version: "3.13"
100             - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
101               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
102             - name: Golden-fixtures characterization (offline)
103               run: |
104                 python -m backend.eval.golden_fixtures --check
105                 python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
106

```

4. user (2026-06-25T17:11:17.247Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11     concurrency:
12         group: ci-${{ github.ref }}
13         cancel-in-progress: true
14
15     jobs:
16         # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17         # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18         # tracked .py, including scripts/ and training/, which the backend-only
19         # py_compile smoke sweep does not reach.
20         lint:
21             name: Ruff lint
22             runs-on: ubuntu-latest
23             steps:
24                 - uses: actions/checkout@v6
25                 - uses: actions/setup-python@v6
26                   with:
27                     python-version: "3.13"
28                 - name: Install ruff
29                   run: python -m pip install ruff
30                 - name: Ruff check
31                   run: ruff check . --output-format github
32
33         # PII / attribution leak guard (tools/leak_scan.py): fails on a committed 32-hex MD5
34         # (the video_id shape) or a personal absolute user path in the shipped-code surface

```

```

35     # (backend/training/scripts/tools/eval_baselines; tests/ excluded ? legit fixtures).
Real
36     # names are scanned only via a gitignored private list locally; CI runs the structural
guard.
37     leak-scan:
38         name: Leak scan (PII / attribution guard)
39         runs-on: ubuntu-latest
40         steps:
41             - uses: actions/checkout@v6
42             - uses: actions/setup-python@v6
43               with:
44                 python-version: "3.13"
45             - name: Leak scan
46               run: python -m tools.leak_scan
47
48     # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
49     # per-module quarantine list ? ratchet by removing entries as modules clean
50     # up. Typed core deps are installed so mypy sees their real signatures
51     # (without them ignore_missing_imports would Any-stub half the checks).
52     typecheck:
53         name: Mypy (lenient baseline)
54         runs-on: ubuntu-latest
55         steps:
56             - uses: actions/checkout@v6
57             - uses: actions/setup-python@v6
58               with:
59                 python-version: "3.13"
60             - name: Install mypy + typed core deps
61               run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
62             - name: Mypy
63               run: mypy backend training
64
65     # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
66     # module-level import error shipping while the suite stays green.
67     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
68     # on machines without the GPU stack (test_import_smoke stubs what's absent).
69     import-smoke:
70         name: Syntax sweep + import smoke (no GPU stack)
71         runs-on: ubuntu-latest
72         steps:
73             - uses: actions/checkout@v6
74             - uses: actions/setup-python@v6
75               with:
76                 python-version: "3.13"
77             - name: Install minimal deps
78               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
79             - name: Run import smoke tests
80               run: python -m pytest tests/test_import_smoke.py -v
81
82     full-suite:
83         name: Full test suite (offline, mocked)
84         runs-on: ubuntu-latest
85         steps:
86             - uses: actions/checkout@v6
87             - uses: actions/setup-python@v6
88               with:
89                 python-version: "3.13"
90                 cache: pip
91             - name: System libs for opencv-python
92               run: |
93                 sudo apt-get update
94                 sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev

```

```

95     - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
96     # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
97     # need an out-of-band index that PEP 621 can't express. Keep installing
98     # them separately here, before the editable install pulls the rest.
99     run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
100     - name: Install package (editable) + dev tooling
101     # Editable PEP 621 install: runtime deps + the `dev` group
102     # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
103     run: python -m pip install -e .[dev]
104     # obsreport (private, soft dep) is absent here: reporting tests skip via
105     # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
106     # is therefore calibrated against THIS lane's number (local runs with
107     # obsreport installed measure a little higher).
108     # Floor calibrated 2026-06-11 against this lane's measured 72% (local
109     # with obsreport: 73%). Ratchet it UP as coverage grows; never down
110     # without an owner decision.
111     - name: Run full suite (with coverage floor)
112     run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
113
114     # Cross-OS determinism guard for the committed golden regression fixtures
115     # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
116     # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
117     # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
118     # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
119     # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
120     golden-crossos:
121       name: Golden fixtures cross-OS (${{ matrix.os }})
122       strategy:
123         fail-fast: false
124         matrix:
125           os: [ubuntu-latest, windows-latest, macos-latest]
126       runs-on: ${{ matrix.os }}
127       steps:
128         - uses: actions/checkout@v6
129         - uses: actions/setup-python@v6
130         with:
131           python-version: "3.13"
132         - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
133         run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
134         - name: Golden-fixtures characterization (cross-OS determinism guard)
135         # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
(synthetic /
136         # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
the
137         # de-attribution paths get cross-OS coverage and surface any win/mac float
drift.
138         run: |
139           python -m backend.eval.golden_fixtures --check
140           python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
141
142     # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d).
Scans the
143     # RUNTIME dependency closure (`pip install .` ? no torch/dev) and fails on any
copyleft
144     # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band and is not in this
closure. This
145     # automates a guardrail that was human-only; the report is printed for review either
way.
146     license-scan:
147       name: Dependency license scan (no copyleft in runtime deps)
148       runs-on: ubuntu-latest

```

```

149     steps:
150     - uses: actions/checkout@v6
151     - uses: actions/setup-python@v6
152       with:
153         python-version: "3.13"
154     - name: Install runtime deps + pip-licenses
155       run: |
156         python -m pip install .
157         python -m pip install pip-licenses
158     - name: Report dependency licenses
159       run: python -m piplicenses --format=markdown --with-urls --order=license
160     - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
161       run: |
162         if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
163           echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL)
license ? the guardrail is MIT/Apache/BSD only. Review the report above."
164           exit 1
165         fi
166         echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."
167
168     # Built-wheel smoke (PACKAGING_PLAN.md P0d): build the wheel, install it NON-editable
from a
169     # non-repo cwd, and launch the console script. Catches packaging regressions the
source suite
170     # can't ? namespace-package resolution (backend has no __init__.py), missing data
files, hidden
171     # imports. (Validated locally: a clean-venv wheel install + CPU torch runs `indexer
--help`.)
172     build-artifact:
173       name: Built wheel smoke (install + launch)
174       runs-on: ubuntu-latest
175       steps:
176       - uses: actions/checkout@v6
177       - uses: actions/setup-python@v6
178         with:
179           python-version: "3.13"
180       - name: System libs for opencv-python
181         run: |
182           sudo apt-get update
183           sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
184       - name: Build the wheel
185         run: |
186           python -m pip install build
187           python -m build --wheel
188       - name: Install the built wheel + CPU torch (out-of-band index)
189         run: |
190           python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
191           python -m pip install dist/*.whl
192       - name: Launch the installed console script from a non-repo dir
193         run: |
194           cd /tmp
195           indexer --help
196           python -c "import backend, training, backend.utils.app_home; print('wheel
import OK')"
197

```

5. assistant (2026-06-25T17:11:20.163Z)

Now let me verify the claims about the golden_fixtures module, the `--check` flag, and the referenced test files. Let me check these in parallel.

6. user (2026-06-25T17:11:22.677Z)

```
-rw-r--r-- 1 avidu 197609 42975 Jun 25 16:24 C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/golden_fixtures.py
---tests---
-rw-r--r-- 1 avidu 197609 7293 Jun 25 16:24 C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_golden_fixtures.py
-rw-r--r-- 1 avidu 197609 28710 Jun 25 16:24 C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_golden_real_fixtures.py
```

7. user (2026-06-25T17:11:24.148Z)

```
-rw-r--r-- 1 avidu 197609 1999 Jun 25 16:24 C:/Users/avidu/Projects/badminton-highlight-indexer/mypy.ini
-rw-r--r-- 1 avidu 197609 4525 Jun 25 16:24 C:/Users/avidu/Projects/badminton-highlight-indexer/pyproject.toml
-rw-r--r-- 1 avidu 197609 809 Jun 25 16:24 C:/Users/avidu/Projects/badminton-highlight-indexer/ruff.toml
-rwxr-xr-x 1 avidu 197609 2055 Jun 7 14:52 C:/Users/avidu/Projects/badminton-highlight-indexer/run_all_tests.py
```

8. assistant (2026-06-25T17:11:26.606Z)

All referenced files exist. Now let me verify the `--check` flag exists in `golden_fixtures.py` and confirm it has a `\_\_main\_\_` entry point so `python -m backend.eval.golden_fixtures` works.

9. user (2026-06-25T17:11:28.247Z)

```
52:import argparse
726:def main(argv: Optional[Sequence[str]] = None) -> int:
727:    ap = argparse.ArgumentParser(
730:        g.add_argument("--freeze-synthetic", action="store_true",
733:        g.add_argument("--refresh-snapshot", action="store_true",
735:        g.add_argument("--check", action="store_true",
737:        g.add_argument("--freeze-real", action="store_true",
740:        ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate slug for "
742:        ap.add_argument("--all", action="store_true", help="all slugs for --refresh-snapshot")
743:        ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot (review the diff!)")
745:        ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV (Frame,Visibility,X,Y)")
746:        ap.add_argument("--labels", help="--freeze-real) source golden labels CSV (start,end)")
747:        ap.add_argument("--license", dest="license_", default=None,
749:        ap.add_argument("--owned", action="store_true",
751:        ap.add_argument("--minors-free", dest="minors_free", action="store_true",
753:        ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
754:        ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-real) source frame width (px)")
755:        ap.add_argument("--label", default="cleared_real",
758:        ap.add_argument("--source-note", dest="source_note", default="",
762:        ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
764:        ap.add_argument("--max-frames", dest="max_frames", type=int, default=None,
768:        ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
822:        # --check
851:if __name__ == "__main__":
```

10. assistant (2026-06-25T17:11:31.066Z)

The `--check` flag exists (line 735) and there's a `\_\_main\_\_` block (line 851). Let me verify the `main()` is wired to `__main__` and check the `--check` branch behavior (exit code).

11. user (2026-06-25T17:11:31.971Z)

```
818         print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
```

```

to the "
819             "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
820         return 0
821
822     # --check
823     return _cmd_check(fixtures_dir)
824
825
826     # =====
827     # Re-exports ? the real-fixture subsystem (P1 + M2) lives in ``golden_real_fixtures``
but
828     # is exposed here so every historical ``golden_fixtures.<name>`` import path stays
valid.
829     # =====
830
831     #: The names the real-fixture module owns; re-exported lazily (PEP 562 ``__getattr__``)
so the
832     #: ``golden_fixtures`` ? ``golden_real_fixtures`` cycle resolves regardless of import
order ?
833     #: ``golden_real_fixtures`` imports the shared primitives FROM this module, so an EAGER
bottom-of-
834     #: file re-export would fail when ``golden_real_fixtures`` is imported FIRST (its own
names would
835     #: not yet be defined when this module's import of them runs). The lazy hook defers the
import to
836     #: first attribute access, by which point BOTH modules are fully initialized.
837     _REAL_FIXTURE_EXPORTS: Tuple[str, ...] = (
838         "RefusalError", "freeze_real_fixture", "_scan_forbidden", "_new_surrogate_slug",
839         "_reset_trajectory_rows", "_reset_labels", "_SCHEMA_FILENAME_LITERALS",
840     )
841
842
843     def __getattr__(name: str) -> Any:
844         """Lazily resolve the re-exported real-fixture names from ``golden_real_fixtures``
(PEP 562)."""
845         if name in _REAL_FIXTURE_EXPORTS:
846             from backend.eval import golden_real_fixtures
847             return getattr(golden_real_fixtures, name)
848             raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
849
850
851     if __name__ == "__main__":
852         raise SystemExit(main())

```

12. assistant (2026-06-25T17:11:35.282Z)

The ``--check`` invocation is correct and wired (`__main__` ? `_cmd_check`, `SystemExit(main())`). Now let me verify the `pyproject.toml` for the `[dev]` extra (`run_all_tests.py` deps), and the `ruff.toml`/`mypy.ini` targets match. Let me also confirm `run_all_tests.py` accepts the `--cov-*` passthrough flags.

13. user (2026-06-25T17:11:36.253Z)

```

1     # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2     #
3     # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4     # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5     # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6     # auto-detection would otherwise miss the un-__init__.d `backend` root.
7     #
8     # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9     # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10    # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]

```



```

11 # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12 [build-system]
13 requires = ["hatchling"]
14 build-backend = "hatchling.build"
15
16 [project]
17 name = "badminton-highlight-indexer"
18 version = "0.1.0"
19 description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20 readme = "README.md"
21 requires-python = ">=3.10"
22 license = { text = "MIT" }
23 authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24 keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26 # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27 dependencies = [
28     "fastapi>=0.111.0",
29     "uvicorn>=0.30.1",
30     "opencv-python>=4.9.0.80",
31     "numpy>=1.26.4",
32     "pydantic>=2.7.1",
33     "jinja2>=3.1.4",
34     "python-dotenv>=1.0.0",
35     "google-genai>=2.4.0",
36     "supervision>=0.21.0,<0.30.0",
37     # Used directly on the serving path (backend/features/rally_seq.py: uniform_filter1d
38     # maximum_filter1d); previously present only TRANSITIVELY via supervision, so a
39     # bump or a pruned-transitive freeze could silently break inference. Declare it.
40     "scipy>=1.10",
41 ]
42
43 [project.optional-dependencies]
44 # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
45 dev = [
46     "pytest",
47     "pytest-cov",
48     "httpx",          # fastapi TestClient transport
49     "ruff",
50     "mypy",
51     "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
52 ]
53
54 # M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
55 security.edge_auth_enabled=True.
56 # `jwt` is lazy-imported, so the base runtime works without this; install it for
57 # hosted/multi-tenant.
58 auth = ["PyJWT[crypto]>=2.8"]
59
60 # Per-video observability reports (SOFT dependency: the pipeline degrades to a
61 # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
62 # fetched over SSH; if you lack access, simply omit this group ? reporting
63 # tests skip via pytest.importorskip. For local dev against a sibling checkout:
64 # pip install -e ../sports-obsreport --no-build-isolation
65 reporting = [
66     "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
67 ]
68
69 # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
70 # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
71 # express, so they are NOT declarable here. Install them out-of-band instead:
72 # # CUDA 12.4 box:

```

```

71 # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
72 # # CPU-only box (also what CI uses):
73 # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
74 gpu = []
75
76 # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
77 # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
78 # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
79 # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
80 # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
81 storage-s3 = ["boto3>=1.34"]
82 storage-gcs = ["google-cloud-storage>=2.16"]
83
84 [project.scripts]
85 # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
86 indexer = "backend.main:main"
87
88 [tool.hatch.metadata]
89 # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
90 # reference*); hatchling requires opting in to allow those in metadata.
91 allow-direct-references = true
92
93 [tool.hatch.build.targets.wheel]
94 # `backend` is a namespace package (no __init__.py); list both import roots
95 # explicitly so an editable/wheel install exposes `import backend` and
96 # `import training` regardless of cwd.
97 packages = ["backend", "training"]
98

```

14. user (2026-06-25T17:11:36.394Z)

```

1 #!/usr/bin/env python
2 """Run the FULL test suite for the badminton-highlight-indexer.
3
4 A single, discoverable entry point so you (or a reviewer / CI) can verify the
5 whole suite before opening a PR:
6
7     python run_all_tests.py          # full suite, quiet
8     python run_all_tests.py -v       # verbose (passthrough to pytest)
9     python run_all_tests.py -k config # filter by keyword (passthrough)
10    python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
11
12 Anything after the script name is passed straight through to pytest.
13
14 Notes
15 -----
16 - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17 - The observability **reporting** tests skip cleanly if the optional ``obsreport``
18   package isn't installed (they use ``pytest.importorskip``). To exercise them too:
19     pip install -e ../sports-obsreport --no-build-isolation
20 - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21 """
22 from __future__ import annotations
23
24 import importlib.util
25 import subprocess
26 import sys
27
28
29 def _installed(module: str) -> bool:
30     try:
31         return importlib.util.find_spec(module) is not None

```

```

32         except (ImportError, ValueError):
33             return False
34
35
36     def main() -> int:
37         args = list(sys.argv[1:])
38         # Default to quiet unless the caller chose a verbosity.
39         if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
40             args = ["-q", *args]
41         cmd = [sys.executable, "-m", "pytest", *args]
42
43         obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests
will SKIP"
44         bar = "=" * 72
45         print(bar)
46         print(" Full test suite: " + " ".join(cmd))
47         print(f" obsreport (optional reporting dep): {obs}")
48         print(bar, flush=True)
49
50         rc = subprocess.run(cmd).returncode
51
52         print(bar)
53         print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}"))
54         print(bar)
55         return rc
56
57
58     if __name__ == "__main__":
59         raise SystemExit(main())
60

```

15. assistant (2026-06-25T17:11:42.090Z)

Now I have a complete picture. Let me verify a couple more things: the cron expression validity, the latest actions versions (the prompt says "should be Node-24, e.g. checkout@v5+ not @v4" ? but the files use @v6, so I should check whether v6 actually exists / is the issue), and confirm there's no `import-smoke` / `leak-scan` / `license-scan` lane being claimed as mirrored (nightly omits those, which is fine since it's "offline lanes" ? but import-smoke and leak-scan ARE offline). Let me re-examine what nightly claims vs what ci.yml has.

Let me check the Python version requirement consistency and whether `mypy backend training` requires deps that the typecheck lane installs (e.g., does mypy need `jinja2`, `scipy`, `supervision`?).

16. user (2026-06-25T17:11:42.596Z)

```

1     [mypy]
2     # Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
3     # modules and pins an explicit quarantine list below ? not a typing crusade.
4     # Ratchet: when you touch a quarantined module, try removing its entry.
5     ignore_missing_imports = True
6     # backend/ is a namespace package (no root __init__.py) ? required so mypy
7     # maps files to backend.* module names instead of duplicating top-level names.
8     explicit_package_bases = True
9
10    # --- Quarantine (known-loose modules, each with a reason) ---
11
12    # Optional module globals (db_instance/global_config set by create_app() / main()).
13    # Route handlers use request.app.state exclusively; helpers take explicit params.
14    # backend.api.job_worker lifted from quarantine (now takes explicit config/db params).
15    [mypy-backend.main]
16    ignore_errors = True
17
18    # Worker dispatcher: composes the SAME validator/segmenter/report building blocks as

```

```

19 # backend.main, which are typed `config: dict` but receive the AppConfig migration shim
20 # at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine
21 # until the typed-config migration removes the dict?AppConfig duality everywhere.
22 [mypy-backend.worker.__main__]
23 ignore_errors = True
24
25 # yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
26 # to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
27 # here is pre-existing orchestration debt unrelated to that move (implicit-Optional
28 # process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
29 # annotation) ? ratchet those out in a focused typing pass, not here.
30 # (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
31 [mypy-backend.pipeline.segmenters.yolo_hybrid]
32 ignore_errors = True
33
34 [mypy-backend.pipeline.segmenters.gemini]
35 ignore_errors = True
36
37 # Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
38 [mypy-backend.pipeline.segmenters.base]
39 ignore_errors = True
40

```

17. assistant (2026-06-25T17:11:48.264Z)

Now I have everything I need to assess fidelity. Let me note the key observation:
`nightly.yml::typecheck` installs `mypy pydantic fastapi uvicorn python-dotenv google-genai numpy` ? identical to `ci.yml::typecheck`. Good, that's a verbatim match.

Let me do one final verification ? check whether `actions/checkout@v6` and `actions/setup-python@v6` are valid existing major versions (since the prompt asserts they "should be Node-24, e.g. checkout@v5+"). The fact that ci.yml (which has green CI and actually ran) uses @v6 already confirms they resolve. Both files use @v6 consistently. Let me confirm there's no version drift between the two files.

18. user (2026-06-25T17:11:50.332Z)

=== nightly.yml uses ===

```

37: - uses: actions/checkout@v6
38: - uses: actions/setup-python@v6
40:   python-version: "3.13"
53: - uses: actions/checkout@v6
54: - uses: actions/setup-python@v6
56:   python-version: "3.13"
70: - uses: actions/checkout@v6
71: - uses: actions/setup-python@v6
73:   python-version: "3.13"
96: - uses: actions/checkout@v6
97: - uses: actions/setup-python@v6
99:   python-version: "3.13"

```

=== ci.yml uses ===

```

24: - uses: actions/checkout@v6
25: - uses: actions/setup-python@v6
27:   python-version: "3.13"
41: - uses: actions/checkout@v6
42: - uses: actions/setup-python@v6
44:   python-version: "3.13"
56: - uses: actions/checkout@v6
57: - uses: actions/setup-python@v6
59:   python-version: "3.13"
73: - uses: actions/checkout@v6
74: - uses: actions/setup-python@v6
76:   python-version: "3.13"
86: - uses: actions/checkout@v6

```

```

87:      - uses: actions/setup-python@v6
89:        python-version: "3.13"
128:      - uses: actions/checkout@v6
129:      - uses: actions/setup-python@v6
131:        python-version: "3.13"
150:      - uses: actions/checkout@v6
151:      - uses: actions/setup-python@v6
153:        python-version: "3.13"
176:      - uses: actions/checkout@v6
177:      - uses: actions/setup-python@v6
179:        python-version: "3.13"

```

19. assistant (2026-06-25T17:11:58.826Z)

Both files use `actions/checkout@v6`, `actions/setup-python@v6`, and Python `"3.13"` consistently across every job. The prompt's hint that versions "should be checkout@v5+ not @v4" is satisfied ? they're @v6 (Node-24 capable), matching ci.yml exactly. No version drift.

Let me do a final comparison of the four offline lanes that nightly mirrors vs. their ci.yml originals, command-by-command, to confirm verbatim fidelity and identify any missing offline lanes.

Summary of my command-by-command comparison:

```

**lint:** nightly L41-44 vs ci L28-31 ? `python -m pip install ruff` then `ruff check .
--output-format github`. Identical.

```

```

**typecheck:** nightly L57-60 vs ci L60-63 ? install `mypy pydantic fastapi uvicorn python-
dotenv google-genai numpy`, then `mypy backend training`. Identical.

```

```

**full-suite:** nightly L75-84 vs ci L91-112 ? apt libs, CPU torch, `pip install -e .[dev]`,
then `python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-
under=70`. Identical including the floor=70.

```

```

**golden-fixtures:** nightly L100-105 vs ci `golden-crossos` L132-140 ? same deps, same two
commands (`python -m backend.eval.golden_fixtures --check` + the two pytest files). Identical
(nightly correctly runs single-OS, documented at L88-91).

```

Now I'll record my findings.

20. user (2026-06-25T17:12:13.804Z)

Structured output provided successfully

21. assistant (2026-06-25T17:12:32.174Z)

I've completed the fidelity + runnability review of PR #381. Here is my assessment.

Verdict: the workflow is correct and runnable. All four lanes mirror ci.yml faithfully; the steps WILL run.

I verified each concern against the actual files (the prompt's caveat that the PR's green CI only YAML-parsed nightly.yml is correct ? schedule/workflow_dispatch don't self-trigger).

(a) Fidelity vs ci.yml ? verbatim on all 4 mirrored lanes, no drift

- **Python pin:** `"3.13"` on every job, identical to ci.yml.
- **Action versions:** `actions/checkout@v6` + `actions/setup-python@v6` on every job ? same as ci.yml, satisfying the Node-24 requirement (well past the @v5+ floor the prompt flagged).
- **lint:** `pip install ruff` ? `ruff check . --output-format github` ? identical (ci.yml:28-31).
- **typecheck:** installs `mypy pydantic fastapi uvicorn python-dotenv google-genai numpy` ? `mypy backend training` ? identical per-lane deps (ci.yml:60-63), so nothing Any-stubs for a missing dep.
- **full-suite:** apt libs ? CPU-only torch ? `pip install -e .[dev]` ? `python run\_all\_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70` ? identical, **floor=70**

```
preserved** (ci.yml:112).  
- **golden-fixtures:** same minimal deps + same two commands as ci.yml's `golden-crossos`  
(ci.yml:132-140).
```

(b) Runnability of every command ? all valid

```
- `python -m backend.eval.golden_fixtures --check` (the command NOT in ci.yml verbatim, though  
it IS in ci.yml's golden-crossos): module exists at `backend/eval/golden_fixtures.py`; `--check`  
flag is defined (line 735); `__main__` is wired via `raise SystemExit(main())` (line 851-852)  
and `main()` returns `_cmd_check(...)` for `--check` (line 823). Correct invocation, propagates  
exit code.  
- Both pytest target files exist: `tests/test_golden_fixtures.py`,  
`tests/test_golden_real_fixtures.py`.  
- `[dev]` extra exists in pyproject.toml (pytest, pytest-cov, httpx, ruff, mypy, PyJWT).  
`run_all_tests.py` passes args straight through to pytest, so the `--cov-*` flags work.
```

(c) cron `0 3 \* \* \*` ? valid (03:00 UTC daily). Header L14-16 also correctly warns scheduled workflows only fire from the default branch

Findings (both non-blocking)

1. **LOW ? partial offline mirror:** nightly carries 4 lanes but ci.yml has more *offline* lanes
it drops ? notably `leak-scan` and `import-smoke` (both pure-offline, on-mission for drift/rot),
plus `license-scan` (guards the non-negotiable MIT/Apache/BSD rule, exactly the "transitive-dep
drift" the header says nightly exists to catch). The header's "mirror ci.yml's offline jobs
verbatim" overstates scope. Fix: add those lanes, or soften the comment to state which offline
lanes are intentionally in/out and why.
2. **NIT ? single-OS golden lane** vs ci.yml's 3-OS matrix: intentional and clearly documented
(L88-91). No action needed.

```
Files: `C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\nightly.yml`,  
`C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\ci.yml`,  
`C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\golden_fixtures.py`.
```